

[Problem](#) | [Submissions](#) | [Leaderboard](#) | [Discussions](#) | [Editorial](#) 

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The `Player` class is provided for you in your editor. It has 2 fields: a **`name`** `String` and a **`score`** `integer`.

Given an array of `n` `Player` objects, write a comparator that sorts them in order of decreasing score: if 2 or more players have the same score, sort those players alphabetically by name. To do this, you must create a `Checker` class that implements the `Comparator` interface, then write an `int compare(Player a, Player b)` method implementing the `Comparator.compare(T o1, T o2)` method.

Input Format

Input from `stdin` is handled by the locked stub code in the `Solution` class.

The first line contains an integer, `n`, denoting the number of players.

Each of the `n` subsequent lines contains a player's **`name`** and **`score`**, respectively.

Constraints

- $0 \leq \text{score} \leq 1000$
- 2 players can have the same name.
- Player names consist of lowercase English letters.

Output Format

Author [Shafaet](#)
Difficulty [Medium](#)
Max Score 10
Submitted By [121408](#)

NEED HELP?

 [View discussions](#)
 [View editorial](#)
 [View top submissions](#)

RATE THIS CHALLENGE

☆☆☆☆☆

MORE DETAILS

 [Download problem statement](#)
 [Download sample test cases](#)



Upload Code as File



Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
1 12
2 ENTER John 3.75 50
3 ENTER Mark 3.8 24
4 ENTER Shafaet 3.7 35
5 SERVED
6 SERVED
7 ENTER Samiha 3.85 36
8 SERVED
9 ENTER Ashley 3.9 42
10 ENTER Maria 3.6 46
11 ENTER Anik 3.95 49
```

Download



Description Editorial Solutions Submissions

2418. Sort the People

Easy Topics Companies Hint

You are given an array of strings `names`, and an array `heights` that consists of **distinct** positive integers. Both arrays are of length `n`.

For each index `i`, `names[i]` and `heights[i]` denote the name and height of the i^{th} person.

Return `names` sorted in *descending order by the people's heights*.

Example 1:

Input: `names = ["Mary","John","Emma"], heights = [180,165,170]`

Output: `["Mary","Emma","John"]`

Explanation: Mary is the tallest, followed by Emma and John.

Example 2:

Input: `names = ["Alice","Bob","Bob"], heights = [155,185,150]`

Output: `["Bob","Alice","Bob"]`

Explanation: The first Bob is the tallest, followed by Alice and the second Bob.

Constraints:

$n == \text{names.length} == \text{heights.length}$

1 ≤ `names.length` ≤ 100

1 ≤ `heights.length` ≤ 100

1 ≤ `names[i].length` ≤ 10

`names[i]` consists of lowercase and uppercase English letters.

`heights[i]` is a positive integer.

All the values of `heights` are distinct.

Return `names` sorted in descending order by the people's heights.

7 Online

Code

Java Auto

```
1 class Solution {
2     public String[] sortPeople(String[] names, int[] heights) {
3         int nameSize = names.length;
4         int heightsSize = heights.length;
5         String newArr[] = new String[nameSize];
6         HashMap<Integer, String> map = new HashMap<>();
7         for(int i = 0; i < nameSize; i++){
8             map.put(heights[i], names[i]);
9         }
10        List<Integer> ans = new ArrayList<>(map.keySet());
11        ans.sort((h1, h2) -> h2 - h1);
```

Saved

Ln 18, Col 2

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

names =
["Mary","John","Emma"]

heights =
[180,165,170]

Output

Upload Code as File

Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150
```

Download

Your Output (stdout)

```
aleksa 150
amy 100
david 100
```

Change Theme Language Java 8

```
1 import java.util.*;
2
3 class Checker implements Comparator<Player> {
4
5     @Override
6     public int compare(Player a, Player b) {
7
8         // Higher score first
9         if (a.score != b.score) {
10             return b.score - a.score;
11         }
12
13         // Same score -> alphabetical order
14         return a.name.compareTo(b.name);
15     }
16 }
17
18 class Player{
19     String name;
20     int score;
21
22     Player(String name, int score){
23         this.name = name;
24         this.score = score;
25     }
26 }
27
```

Line: 16 Col: 2

Upload Code as File

Change Theme Language Java 8

```
1 > import java.util.ArrayList; ...
2
3 class Student {
4     private int id;
5     private String name;
6     private double cgpa;
7
8     public Student(int id, String name, double cgpa) {
9         this.id = id;
10        this.name = name;
11        this.cgpa = cgpa;
12    }
13
14    public int getID() {
15        return id;
16    }
17
18    public String getName() {
19        return name;
20    }
21
22    public double getCGPA() {
23        return cgpa;
24    }
25 }
26
27 class Priorities {
28
29
```

Line: 99 Col: 2



Upload Code as File

Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Compiler Message

Success

Input (stdin)

```
4
1 5
3 3
2 9
9 6
```

[Download](#)

Your Output (stdout)

```
1.0000000000
```

```

1 # Enter your code here. Read input from STDIN. Print output to STDOUT
2 import sys
3 import math
4
5
6 def solve(m, n):
7     # Total number of non-decreasing sequences:
8     #  $C(n + m - 1, m)$ 
9     total = math.comb(n + m - 1, m)
10
11     # Precompute  $C(n, j)$ 
12     comb_n = [math.comb(n, j) for j in range(min(m, n) + 1)]
13
14     # Precompute:
15     #  $C(n + r - 1, r)$ , for  $r = 0..m$ 
16     comb_remaining = [0] * (m + 1)
17     comb_remaining[0] = 1
18
19     for r in range(1, m + 1):
20         comb_remaining[r] = (
21             comb_remaining[r - 1] * (n + r - 1) // r
22         )
23
24     sum_a = 0
25
26     # A(k) = number of sequences where
27     # every frequency is  $\leq k$ 

```

Line: 67 Col: 11



Upload Code as File



Test against custom input

Run Code

Submit Code

```
4  public class Solution {  
6  public static void main(String[] args) {  
31      // Number of queries  
32      int q = sc.nextInt();  
33  
34      for (int i = 0; i < q; i++) {  
35  
36          int x = sc.nextInt();  
37          int y = sc.nextInt();  
38  
39          // Convert 1-based positions to 0-based indexes  
40          if (x >= 1 &&  
41              x <= data.size() &&  
42              y >= 1 &&  
43              y <= data.get(x - 1).size()) {  
44  
45              System.out.println(data.get(x - 1).get(y - 1));  
46  
47          } else {  
48              System.out.println("ERROR!");  
49          }  
50      }  
51  
52      sc.close();  
53  }  
54  }
```

Change Theme Language Java 8

```
4  public class Solution {  
6  public static void main(String[] args) {  
31      // Number of queries  
32      int q = sc.nextInt();  
33  
34      for (int i = 0; i < q; i++) {  
35  
36          int x = sc.nextInt();  
37          int y = sc.nextInt();  
38  
39          // Convert 1-based positions to 0-based indexes  
40          if (x >= 1 &&  
41              x <= data.size() &&  
42              y >= 1 &&  
43              y <= data.get(x - 1).size()) {  
44  
45              System.out.println(data.get(x - 1).get(y - 1));  
46  
47          } else {  
48              System.out.println("ERROR!");  
49          }  
50      }  
51  
52      sc.close();  
53  }  
54  }  
55  }
```

Line: 54 Col: 2